

Laboratório 05

Parte 1

Neste laboratório o objetivo é implementar um microcontrolador simples de 16 bits. O laboratório está dividido em 5 partes, e neste documento iremos detalhar apenas a primeira destas.

Parte 1

Nesta parte você deve implementar o circuito da Figura 1. Observe o uso de componentes recorrentes, como registradores com sinais de enable, multiplexadores e somadores. Por ora, iremos implementar apenas 4 instruções.

1. 000xxxxxyy: mv Rx,Ry
2. 001xxxddd: mvi Rx,#D
3. 010xxxxxyy: add Rx,Ry
4. 011xxxxxyy: sub Rx,Ry

onde ddd indica entradas não utilizadas (*don't care*).

Três pontos são importantes. Estes 9 bits da instrução são sempre os 9 bits mais significativos de DIN. Segundo, todas as instruções serão executadas em 2 ou 4 ciclos de relógio, e, portanto, estes 9 bits devem ser armazenados através do registrador IR durante o primeiro ciclo. Finalmente, a instrução mvi precisa de um dado externo (#D), que será fornecido também via DIN no segundo ciclo de relógio.

Vamos nos concentrar apenas na unidade de controle. Em todas as condições, inicialmente devemos utilizar o registrador IR para armazenar os 9 bits mais significativos da entrada externa DIN. Estamos efetuando o *fetch* da instrução. Assim, neste instante, apenas o sinal IRin deverá ser ativado pela unidade de controle.

A partir do 2º ciclo, a unidade de controle deve adaptar quais sinais ativar a partir da instrução e do ciclo que ciclo o processo se encontra. A Tabela 2 apresenta estes sinais. Verifique, por exemplo, que a operação de soma exige 4 ciclos de relógio. No 1º, é realizado o *fetch* como descrito anteriormente. O 2º ciclo realiza a cópia de Rx no registrador temporário A. O 3º ciclo realiza a soma de A com Ry, armazenando o valor no outro registrador temporário, G. Finalmente, o 4º ciclo copia o conteúdo de G para Rx, terminando a operação.

A forma mais simples de se projetar a máquina de estados é considerar que temos, no caso em questão, quatro estados (T0, T1, T2, T3) indicando em qual ciclo o sistema se encontra. Como a instrução ficará armazenada em IR durante toda a execução da instrução, não é necessário que o estado armazene esta informação. Assim, o 1º process apenas efetua a atualização do estado atual para o estado futuro na subida do clock.

O 2º process deverá construir, de forma combinatória, o próximo estado a partir do estado atual e de entradas. Primeiramente, observe que as transições T0->T1, T2->T3 e T3->T0 independem de qualquer entrada. Apenas a T1->T0 ou T1->T2 dependem da instrução que está sendo executada.

Finalmente, a saída da máquina de estados, que de fato vai implementar todos os sinais de controle, pode ser construída conforme o modelo a seguir:

```
fsmoutput: PROCESS (Tstep_Q, I, Xreg, Yreg)
BEGIN
    Done <= '0'; Ain <= '0'; Gin <= '0'; Gout <= '0'; AddSub <= '0';
    IRin <= '0'; DINout <= '0'; Rin <= "00000000"; Rout <= "00000000";
    CASE Tstep_Q IS
        WHEN T0 =>
            IRin <= '1';
        WHEN T1 =>
            CASE I IS
                WHEN mv =>
                    Rout <= Yreg;
                    Rin <= Xreg;
                    Done <= '1';
            ...
    
```

onde, por exemplo, mv foi definido como a constante

```
CONSTANT mv : std_logic_vector(2 downto 0) := "000";
```

e Xreg, Yreg são as saídas de um decodificador 3 para 8 dos dados dos registradores de entrada e saída dadas na instrução.

Para questão de simulação, iremos implementar apenas o processador, sem conectar a periféricos da placa. Desta forma, a entidade a ser usada é:

```
ENTITY proc IS
    PORT ( DIN           : IN           STD_LOGIC_VECTOR(15 DOWNT0 0);
          Resetn, Clock, Run : IN           STD_LOGIC;
          BusWires         : BUFFER      STD_LOGIC_VECTOR(15 DOWNT0 0);
          Done             : BUFFER      STD_LOGIC);
END proc;
```

onde Clock e Reset são sinais conectados a todos os registradores e à máquina de estados, DIN é a entrada de instrução (e dados no mvi), BusWires é o barramento principal e Done é o sinal indicando que a máquina de estados pode voltar a realizar o *fetch* no próximo ciclo de relógio. Estes últimos dois sinais são do tipo Buffer, o que indica que para efeito de entidade eles são uma saída, porém cujos valores também podem ser lidos interiormente ao circuito.

Para efetuar a simulação, incluímos as diretrizes

```
attribute keep : boolean;
attribute keep of R0,R1,R2,R3,R4,R5,R6,R7,A,G: signal is true;
```

para sermos capazes de observar a saída dos registradores.

Para validar o circuito, na simulação a seguir, efetuamos as seguintes operações

1. mvi R0,#1
2. mv R1,R0
3. add R1,R0

4. add R1,R0

5. sub R1,R0

que forneceu a seguinte simulação. Podemos observar as mudanças dos valores dos registradores R0 e R1 conforme o desejado no programa.

